Informe de Pràctica: Copilot Professional

Assignatura: Mòdul Propi - Big Data

Alumne: Fabio Canaza

Professor: Albert Guardiola

Data d'entrega: 21/4/2026

Introducció

L'objectiu d'aquesta activitat és explorar les capacitats de GitHub Copilot Pro en els seus diferents modes: Inline, Chat i Agent. S'ha treballat sobre un sistema de login existent que presenta mancances en validació, seguretat i arquitectura.

A. Mode Inline Avançat (Autocompletat)

En aquesta secció hem comprovat com el mètode *inline* és eficaç per a tasques ràpides i puntuals, tot i que no dissenya arquitectures complexes.

A1. Generació de handler amb validació

S'ha utilitzat el comentari // handle login with validation before sending request per ampliar el handler del frontend. Copilot ha generat automàticament la comprovació de camps buits i la gestió de missatges d'error abans de fer el fetch.

```
// handle login with validation before sending request
async function handleLoginSubmit(event) {
    event.preventDefault();

    const { email, password } = getLoginFormData();
    const validation = validateLogin(email, password);

    if (!validation.valid) {
        alert(validation.message);
        return;
    }

    try {
        const response = await sendLoginRequest(email, pass
        console.log('Login response:', response);
    } catch (error) {
        alert('Login request failed. Please try again.');
        console.error('Login error:', error);
```

A2. Generació de REGEX

Hem demanat una expressió regular per a una contrasenya forta (8 caràcters, 1 majúscula i 1 número). Copilot ha generat la constant regex corresponent.

```
// function to validate password using regex
function validatePassword(password) {
    // regex for strong password: min 8 chars, 1 uppercase, 1 number
    const regex = /^(?=.*[A-Z])(?=.*\d).{8,}$/;
}
```

A3. Tests unitaris amb Vitest

Mitjançant comentaris, hem generat un fitxer login.test.js per provar casos de fallada (email buit, password incorrecte) i d'èxit. S'ha configurat l'entorn amb npm install vitest.

```
PS D:\Users\alumn\Documents\DAW 2\Mòdul professional optatiu\MPROPI-U3-AC2-Fabio_Canaza> npm run test
 DEV  v4.1.4 D:/Users/alumn/Documents/DAW 2/Mòdul professional optatiu/MPROPI-U3-AC2-Fabio_Canaza

 ✓ login.test.js (3 tests) 11ms
   ✓ Login Validation (3)
     ✓ should fail with empty email 4ms
     ✓ should fail with wrong password 1ms
     ✓ should succeed with valid credentials 1ms

  Test Files  1 passed (1)
       Tests  3 passed (3)
    Start at  18:06:28
    Duration  452ms (transform 37ms, setup 0ms, import 73ms, tests 11ms, environment 0ms)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

B. Mode Chat Avançat (Context Real)

Hem explorat com el context (selecció manual i @file) millora la precisió de les respostes de l'IA.

B1 i B2. Explicació i Refactorització

Seleccionant el codi amb el ratolí, hem demanat una explicació línia a línia i una refactorització per separar la lògica de validació de la de xarxa.

```
// function to validate login input and return detailed error messages
function validateLogin(email, password) {
    if (!email || email.trim() === '') {
        return { valid: false, message: 'Email is required.' };
    }
    if (!password) {
        return { valid: false, message: 'Password is required.' };
    }
    if (!validatePassword(password)) {
        return { valid: false, message: 'Password must be at least 8 characters long, contain an uppercase le
    }
    return { valid: true, message: 'Login successful.' };
}
```

Explica aquest codi línia a línia

JS app.js:1-36

Optimizing tool selection...

Aquí tens l'explicació línia a línia del fragment de JS app.js :

1. `document.getElementById('loginForm').addEventListener('submit', function(event) {`
   Busca el formulari amb id `loginForm` i hi afegeix un "listener" per quan es fa `submit`.
2. `event.preventDefault();`
   Evita el comportament per defecte del formulari (recarregar o enviar la pàgina).
3. `const email = document.getElementById('email').value;`
   Llegeix el valor actual del camp d'entrada amb id `email`.
4. `const password = document.getElementById('password').value;`
   Llegeix el valor actual del camp amb id

```
document.getElementById('loginForm').addEventListener('submit', function(event) {
document.getElementById('loginForm').addEventListener('submit', handleLoginSubmit);

async function handleLoginSubmit(event) {
    event.preventDefault();
    const email = document.getElementById('email').value;
    const password = document.getElementById('password').value;

    // handle login with validation before sending
    if (email && password) {
        // Here you would typically send the login data to your server for authentication
        console.log('Logging in with:', { email, password });
        // For demonstration, we will just log the credentials
    } else {
        alert('Please enter both email and password.');

    const { email, password } = getLoginFormData();
    const validation = validateLogin(email, password);

    if (!validation.valid) {
        alert(validation.message);
```

Keep Undo 1 of 3

validi la força de la contrasenya abans del login.

Refactoritza aquest codi separant validació i xarxa

JS app.js:1-15

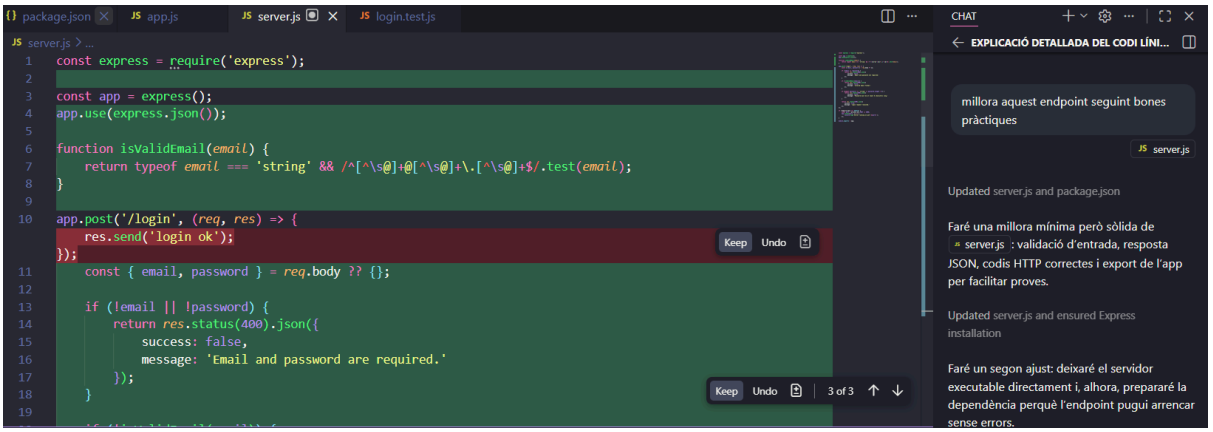
Entenc que vols separar la lògica de validació de la de xarxa al JS app.js ; primer revisaré també els tests per mantenir compatibilitat i després faré el refactor en capes clares (`validació` i `network`).

Reviewed 4 files

Ja tinc el context: JS app.js barreja captura d formulari, validació i "xarxa" (ara mateix un

B3. Millora de Backend amb @file

Utilitzant el símbol @file server.js, hem demanat a Copilot que millori l'endpoint del servidor seguint bones pràctiques.

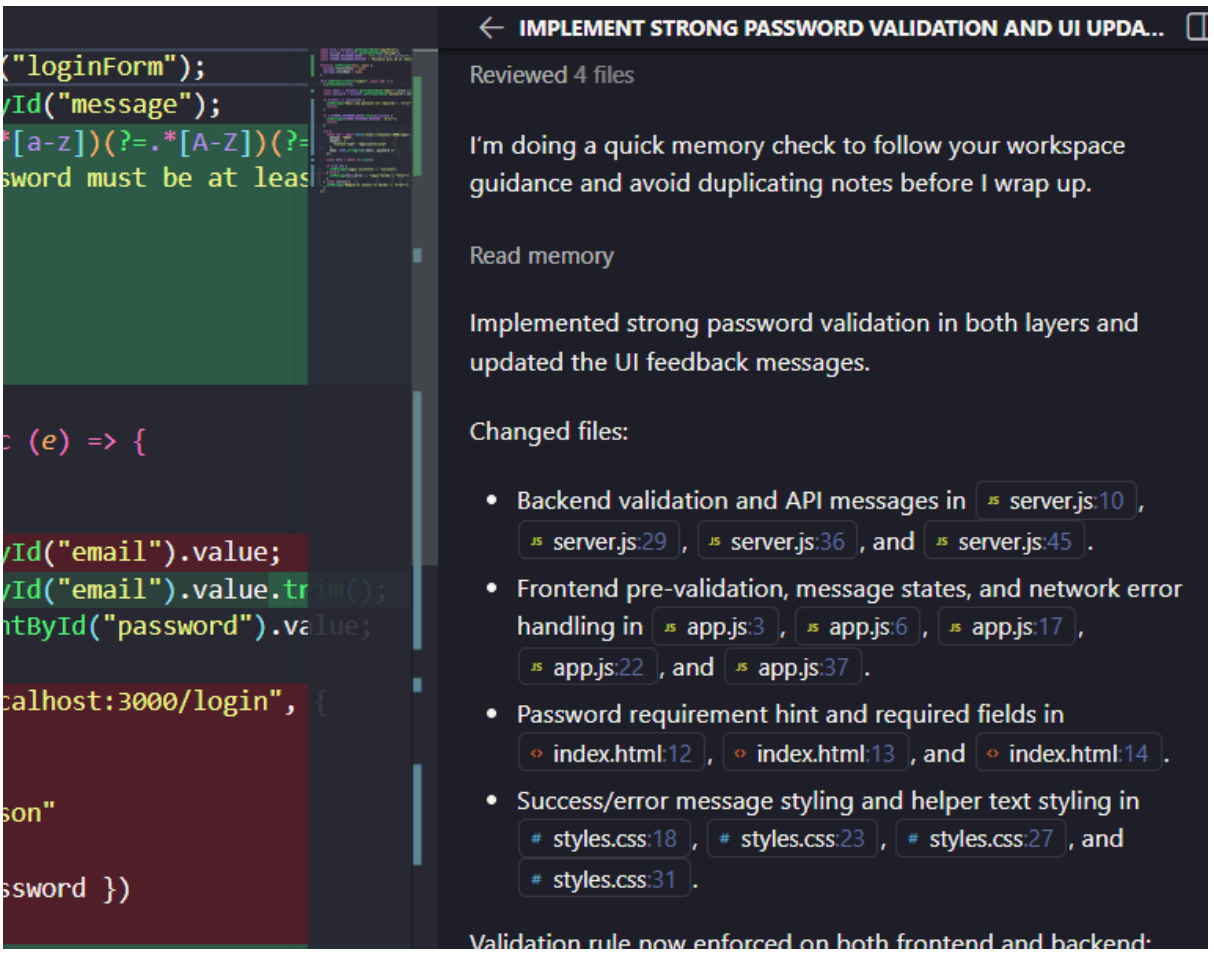


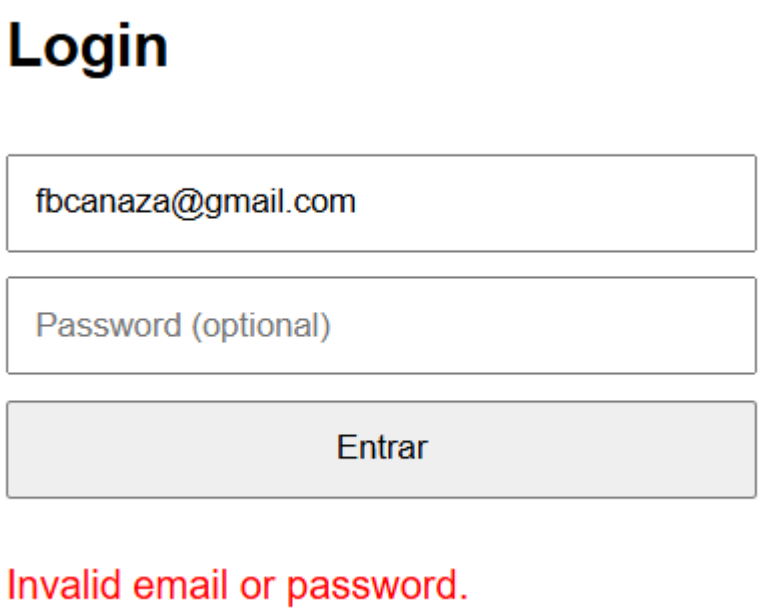
C. Mode Agent

El mode agent permet automatitzacions reals que afecten múltiples fitxers, generant un pla d'acció previ.

C1. Implementació de Feature Completa

Hem demanat afegir validació de contrasenya forta tant al frontend com al backend simultàniament. L'agent ha proposat un pla que modifica diversos fitxers.



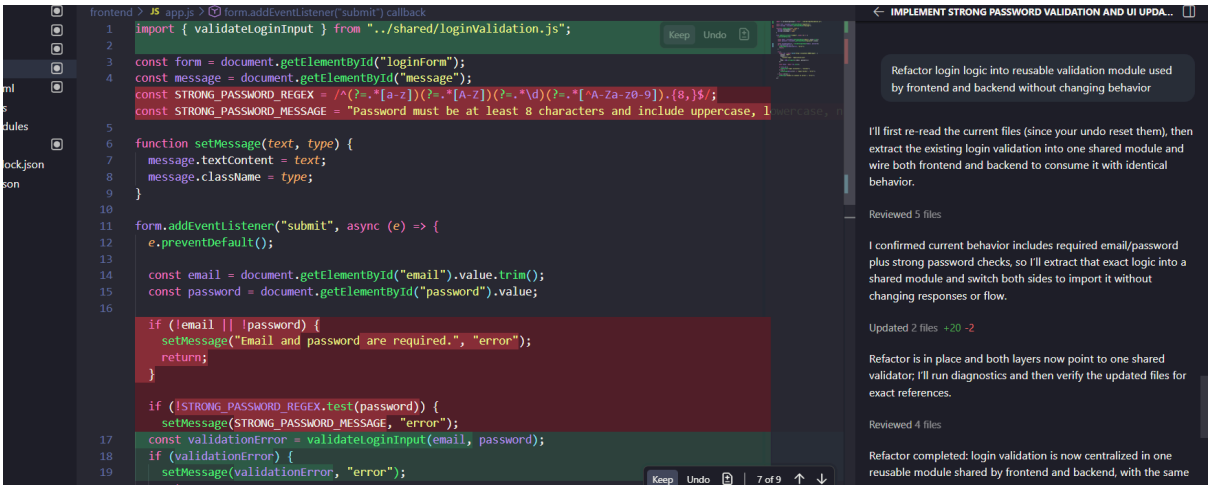


C2. Ús de Checkpoints i Rollback

Hem experimentat amb la seguretat de fer canvis. Després d'aplicar un canvi incorrecte de forma intencionada, hem restaurat el projecte a un estat anterior segur.

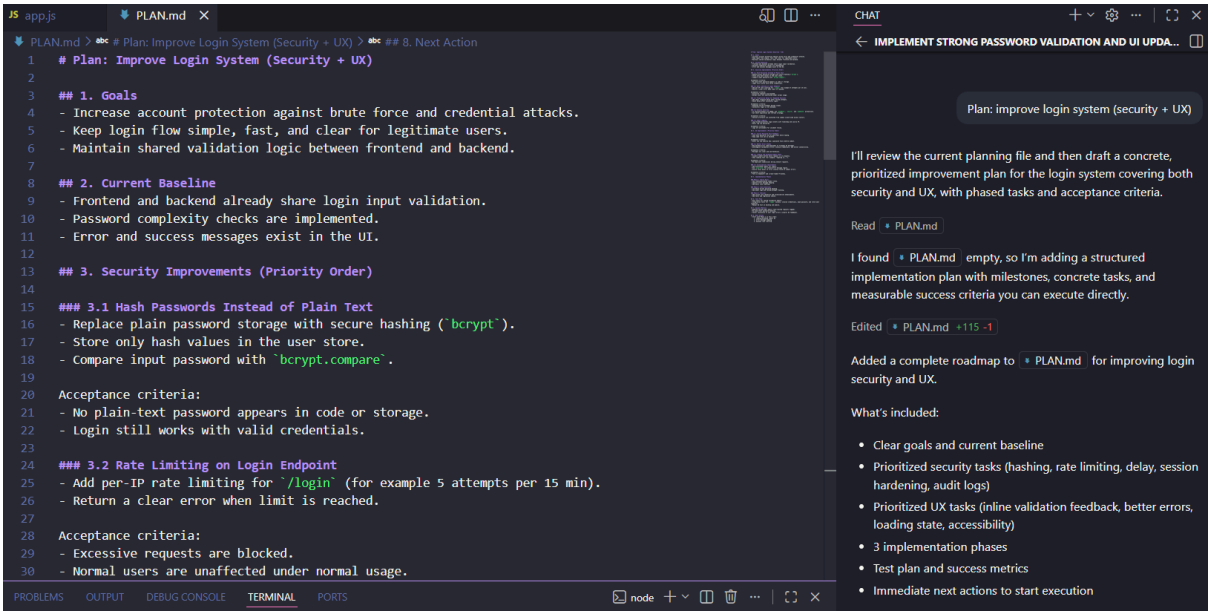
C3. Refactor Multi-arxiu

Hem demanat a l'agent reorganitzar el codi existent sense canviar el seu comportament ("fer-ho millor", no "fer més coses"). L'agent ha extret la lògica a un mòdul reutilitzable validation.js.



E. Mode Planificació

Finalment, hem utilitzat el mode planificació per separar la fase de pensament de la de programació. Hem creat un fitxer PLAN.md amb les millores de seguretat, UX i arquitectura.



Conclusions

Aquesta pràctica ens ha permès entendre que:

- **Inline** és per a execució ràpida.
- **Chat** és per a la reflexió i el disseny.

- **Agent** és per a l'automatització complexa, però requereix supervisió constant per evitar bugs.

Com a desenvolupadors, Copilot no decideix per nosaltres; nosaltres som els responsables de validar i acceptar cada canvi.